FinPilot AI Security Self-Assessment: Permission Tiers, Approval Binding, and Residual Risks

Team member 1
Institution: _____
Email: _____

Team member 2
Institution: _____
Email: _____

***Abstract*—This report assesses the implemented controls and known risks of the FinPilot AI synthetic personal-finance sandbox. It maps assets and trust boundaries to read, proposal, and mutation permissions; describes session ownership, request validation, and exact-payload approval; and connects the controls to reproducible tests. It is an engineering self-assessment, not an independent penetration test, legal compliance opinion, or certification. Only synthetic accounts and financial records are used. Review evidence is dated 19 October 2026; external services and deployment conditions are explicitly distinguished from tested behavior.**

***Index Terms*—agent security, least privilege, approval, auditability, personal finance**

I. Scope and Assets

The assessed system contains the Next.js browser/API boundary, deterministic TypeScript tools, the optional OpenAI planner, the memory/PostgreSQL repository interface, and private FastAPI analytics. Protected assets are each user's ledger, budget/reminder state, session credentials, proposal tokens, model/service keys, and the integrity of audit decisions. Real banks, payment rails, external emails, and investment execution are outside the product. A CSV is treated as synthetic untrusted input regardless of its merchant text.

The threat model considers unauthenticated clients, one authenticated user targeting another, manipulated input or model output, duplicate/racing approval requests, and spreadsheet consumers of exported files. Database administrators, the application host, and the configured TLS gateway remain trusted operators. Their compromise is not prevented by application ownership checks.

II. Permission-Tier Implementation

The model's permission surface consists of narrow read/proposal function schemas. It cannot select an arbitrary SQL operation, HTTP endpoint, filesystem path, or mutation function. Unknown tools and structurally invalid plans do not gain execution authority. Model arithmetic and statements of permission are ignored. A valid but undesirable proposed change still requires human review.

This separation addresses a concrete form of excessive agency described by OWASP [1]. It does not imply that prompt injection is universally solved. The current planner receives the user's question and schemas, not uploaded ledger contents; merchant strings remain inert data in the deterministic executor. Eight tool steps, a 12-second provider timeout, and a 20-second guard bound planning work.

III. Authorization and Atomic Decisions

A random 256-bit proposal token is returned once to its originating browser tab. The server stores its SHA-256 hash, canonical payload hash, owner, five-minute expiry, and status. The originating tab keeps the token in sessionStorage; general ledger responses and conversation history omit token secrets. The decision endpoint validates request shape and Origin, authenticates the session, and compares all proposal bindings. Wrong-user, changed-payload, expired, used, or wrong-token attempts fail.

Successful approval consumes the token, updates the exact reviewed budget/reminder, updates the conversation's decision timeline, and appends an audit record in one versioned ledger update. Rejection consumes the token and appends the decision without modifying financial arrays. A failing operation discards its isolated copy. Concurrent requests cannot both commit the same version; regression tests require exactly one successful use. A denial is logged separately without the rejected token or financial text.

IV. Control Assessment

A. Identity and Ownership

Passwords are bcrypt hashes with cost 12 for registered accounts. Input must have at least 12 characters and at most 72 UTF-8 bytes. One-click demos receive random, unshared passwords and isolated synthetic ledgers. Encrypted iron-session cookies are HttpOnly, SameSite=Lax, and expire after eight hours. Secure is enabled for HTTPS APP_ORIGIN. Session records store a token hash and support immediate logout revocation. Every protected resource lookup derives its owner from this validated session. Caller-supplied user IDs do not select another ledger.

B. Request and Data Validation

Write requests require the expected Origin and JSON content type. No browser wildcard CORS is enabled. Request streaming is capped at 768 KB. Zod constrains dates, months, categories, strings, confirmation values, and integer amounts. Individual money values are bounded; aggregate operations reject unsafe integers. CSV validation rejects an entire invalid

TABLE I
IMPLEMENTED AGENT PERMISSIONS AND AUTHORIZATION BOUNDARY

Tier	Operations	Enforcement
Read	`get_spending_summary`, `list_transactions`, `compare_months`, `get_budget_status`, `detect_recurring_bills`, `explain_anomalies`	Session-derived owner; bounded arguments; source-derived results and audit event. No financial mutation.
Propose	`propose_budget_change`, `propose_bill_reminder`	Server computes an exact payload and issues an expiring random token. Preparing a proposal does not change a budget or reminder.
Mutate	`apply_budget_change`, `apply_bill_reminder`	Not exposed to the model. Verified session, exact payload hash, token hash, pending status, and expiry are required. Decision and financial write commit atomically.
Direct UI	Category editing, CSV commit, budget CRUD, reminder completion	Explicit user-facing confirmation and server validation. These are direct authenticated user commands, separate from the agent's token path.

batch, reruns within the mutation, and skips deterministic duplicates. User strings are escaped by React. Exports neutralize formula-prefixed merchant cells.

C. Secrets, Service Calls, and Headers

Database, model, session, and analytics secrets are read only from server environment variables. No production secret is committed. FastAPI requires its own service token with timing-safe comparison, limits body size to 2 MB and records to 10,000, and has no database or mutation capability. Next.js applies a three-second timeout. CSP, frame denial, nosniff, referrer policy, and disabled device permissions are configured. Protected API responses and downloads use no-store caching.

D. Auditability

Each audit entry contains actor, action, decision, UTC timestamp, correlation ID, and a small explicit metadata object. Passwords, session/approval tokens, API keys, and merchant/chat text are excluded from audit metadata. Payload hashes allow a reviewed proposal to be tied to a recorded decision without storing its token. Audit integrity depends on the ledger store and trusted operators; the implementation is not a separately immutable or externally witnessed log.

V. VERIFICATION EVIDENCE

The full local suites contain 61 TypeScript domain/API checks, 4 Chromium scenarios, and 8 FastAPI checks. Security-relevant evidence includes:

1) Unauthenticated and cross-origin requests are rejected; another account cannot edit a transaction or budget it does not own.
2) Payload modification, cross-user use, expiry, wrong tokens, and replay fail. A wrong-payload attempt does not consume an otherwise valid token.
3) A pair of concurrent approval requests yields one committed application. A mutation throwing after editing its copy leaves stored financial records unchanged.
4) CSV preview performs no write; invalid batches roll back; import requires confirmation; repeated valid imports add no duplicates.
5) Mocked provider output cannot select an apply tool. Unknown/oversized plans and provider failure do not grant write access.
6) Named-merchant validation prevents a request for an unknown merchant from becoming a reminder for a different bill.
7) Browser approval/rejection produces the expected budget and actual audit decision; safe client handling covers network and non-JSON proxy errors.

These checks demonstrate bounded implementation behavior. They do not constitute complete adversarial coverage, a full code-analysis proof, or a real-world detection-quality evaluation. Exact commands and outcomes are recorded in the repository [2].

VI. KNOWN-RISK REGISTER

The register distinguishes implementation controls from deployment work. It supports risk communication in the spirit of NIST AI RMF [3]; it is not an assessment against a legal or banking certification standard. No real financial credentials or money should be introduced under the current evaluation scope.

VII. DEPLOYMENT REVIEW AND SIGN-OFF

Before a durable deployment, verify migrations, database recovery, HTTPS origin, persistent session secret, trusted gateway limits, provider availability, and private service connectivity. Re-run ownership and approval tests against the target repository implementation. Review dependency notices and the point-in-time runtime advisory scan. Do not interpret a green test suite as evidence that all operational or legal requirements are satisfied.

Source and evidence: https://github.com/Shariar-Joy/finpilot-ai. Blank fields are not an executed signature or an authorship attestation.

TABLE II
RESIDUAL RISKS AND APPROPRIATE NEXT CONTROLS

ID	Residual risk	Current boundary	Next control / release condition
R1	In-memory storage loses all accounts and audit history on restart.	Visibly labeled, single-process demo; no silent database fallback.	Use a verified durable PostgreSQL deployment for persistence.
R2	JSONB aggregate rewrites and audit trust do not scale to large datasets.	Owner-scoped compare-and-swap update; copied-state rollback.	Normalize financial rows; separate audit retention and operator protections.
R3	Rate limits are process-local; auth's shared bucket can reduce availability.	API/agent/email buckets and bounded tool/body requests.	Trusted gateway, per-IP/distributed limiting, abuse monitoring.
R4	CSP permits inline framework code/styles.	React escaping, no arbitrary HTML, frame denial.	Per-request nonces and tighter production CSP.
R5	A same-origin script compromise could read proposal tokens.	SessionStorage, five-minute expiry, exact user/payload binding, single use.	Independent XSS assessment and hardened browser policy.
R6	No MFA, verified-email flow, recovery, or account-lifecycle policy.	Password hashing, encrypted cookie, revocation and limits.	Reviewed identity provider and verified recovery before real users.
R7	Model intent selection may be semantically incorrect.	Deterministic values, exact review, no direct model mutation.	Evaluate with a real key and broader intent/adversarial scenarios.
R8	Local live PostgreSQL, external model, and Docker images were not exercised.	Schema/migration validation; mocked provider boundary; real Python service check.	Verify target deployment and read actual CI results before claiming readiness.
R9	Inferred bills and anomaly thresholds can misclassify behavior.	Evidence, explicit labels, no fraud/advice claim.	Labeled evaluation, richer recurrence models, user corrections.
R10	Audit/chat/session arrays and durable demo accounts need retention.	Expiry checked on use; transient demo cap.	Scheduled deletion, retention policy and documented user controls.

TABLE III
TWO-MEMBER REVIEW RECORD TO COMPLETE

	Member 1	Member 2
Name	_____	_____
Review date	_____	_____
Signature	_____	_____

REFERENCES

[1] OWASP GenAI Security Project, "LLM06:2025 excessive agency," https://genai.owasp.org/llmrisk/llm062025-excessive-agency/, 2025, accessed: Oct. 19, 2026.

[2] FinPilot AI Contributors, "FinPilot AI: Source code and reproducibility documentation," https://github.com/Shariar-Joy/finpilot-ai, 2026, see docs/execution-evidence.md and docs/review-report.md for verification scope.

[3] National Institute of Standards and Technology, "Artificial intelligence risk management framework (AI RMF 1.0)," U.S. Department of Commerce, Tech. Rep. NIST AI 100-1, 2023. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf